

bioinformatics.ca

bioinformaticsdotca.github.io

CC BY-SA 4.0 DEED

Attribution-ShareAlike 4.0 International

Canonical URL : https://creativecommons.org/licenses/by-sa/4.0/

See the legal code

You are free to:

Share — copy and redistribute the material in any medium or format for any purpose, even commercially.

Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

Attribution — You must give appropriate credit , provide a link to the license, and indicate if changes were made . You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

ShareAlike — If you remix, transform, or build upon the material, you must distribute your contributions under the same license as the original.

No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Notices:

You do not have to comply with the license for elements of the material in the public domain or where your use is permitted by an applicable exception or limitation .

No warranties are given. The license may not give you all of the permissions necessary for your intended use. For example, other rights such as publicity, privacy, or moral rights may limit how you use the material.

Canadian Bioinformatics Workshops

Model Interpretability ("opening the black box")

Dmytro Tymoshenko

Machine Learning Foundations: Halifax, NS

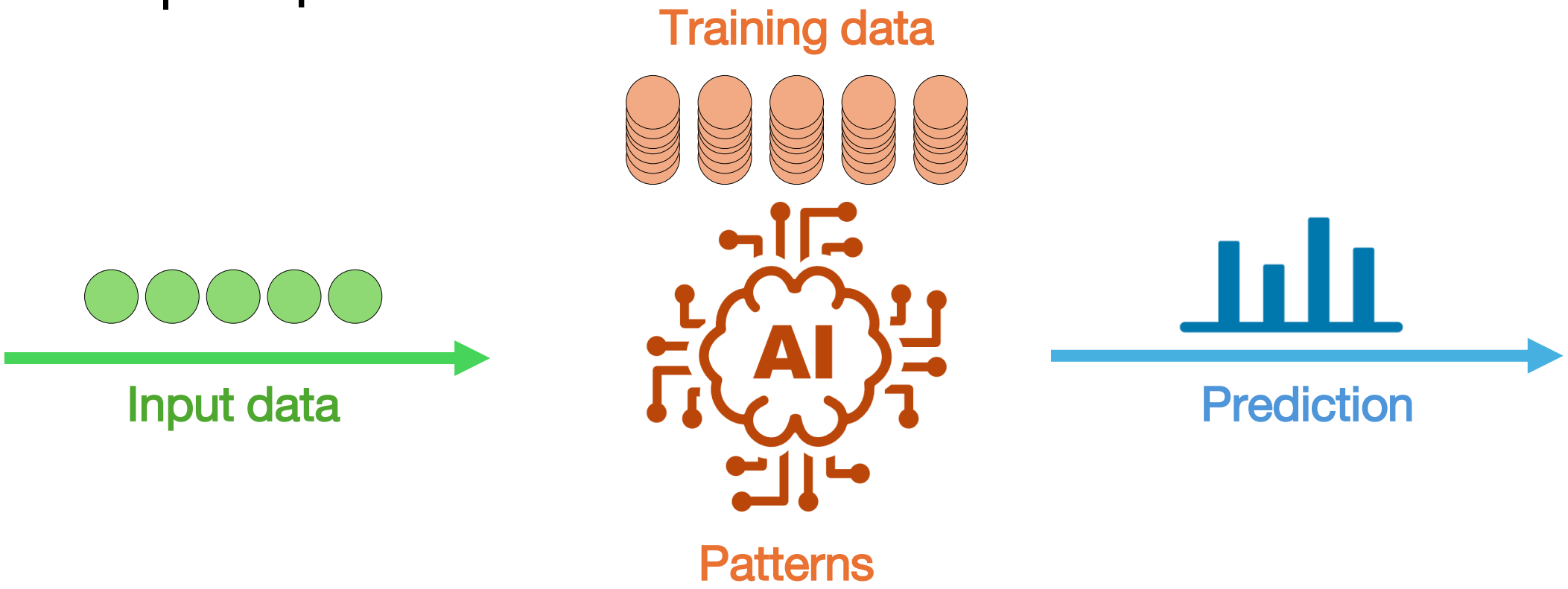
Oct 14 - Oct 16, 2026



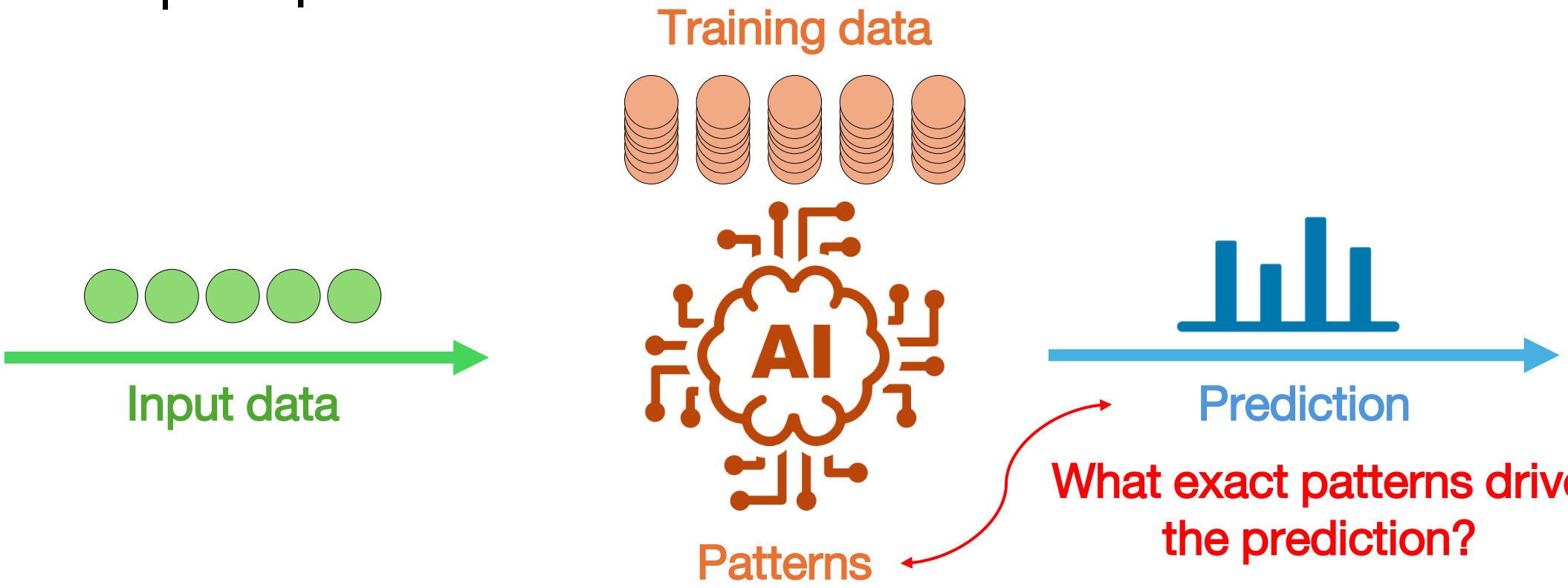
Learning objectives

- Explainable AI framework
- Model-Agnostic / Model-Specific Explainability
- SHAP
- Global / Local Explainability
- Explainability Metrics

XAI | Explainable AI



XAI | Explainable AI



Improved decision-making

Trust and acceptance

Reduced risks and liabilities

Single-case assessments

Complexity vs explainability

Linear/Logistic Regression

Decision Tree based

KNN

SVM

ANN

CNN

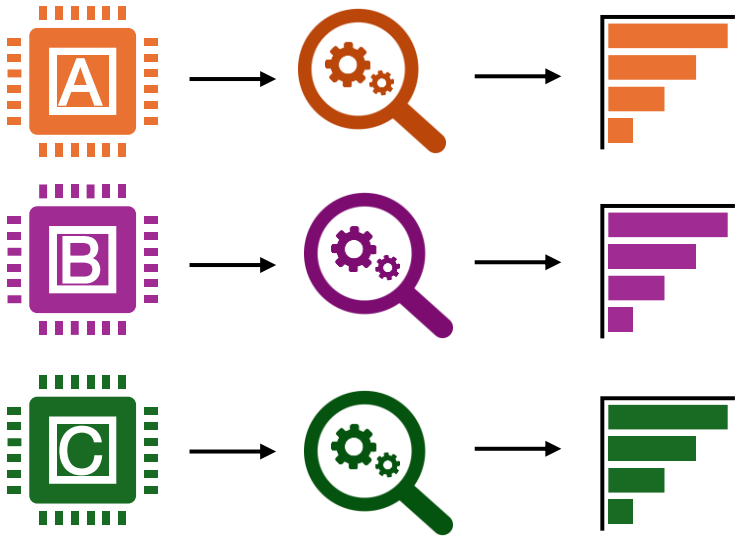
LSTM



Explainable AI framework methods are primary used for interpreting supervised learning models, but it is still can be appliable to any kind of models

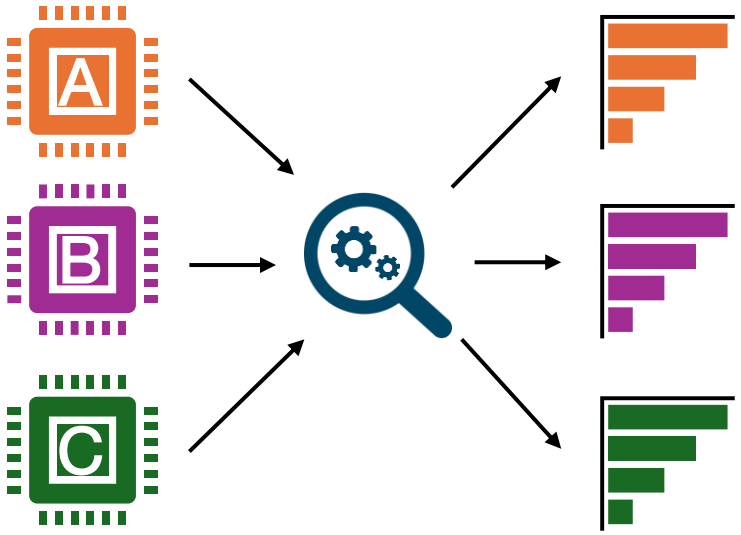
Explainability methods

Model-specific techniques



- specific model families
- utilize existing model data
- cheap, exact and instant

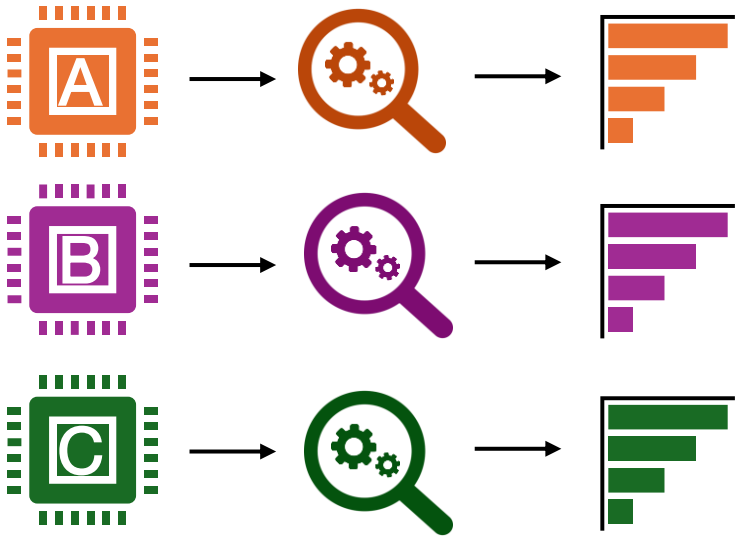
Model-agnostic techniques



- Versatile
- Based on approximation & data
- Computationally heavy

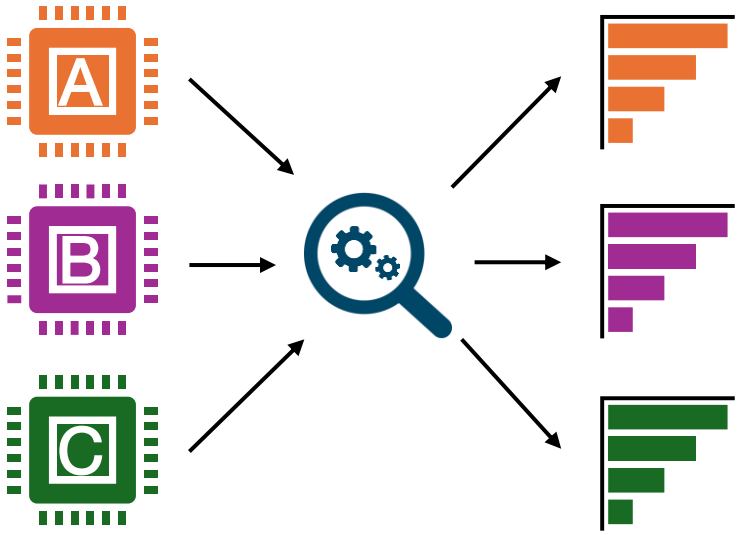
Explainability methods

Model-specific techniques



- specific model families
- utilize existing model data
- cheap, exact and instant

Model-agnostic techniques



- Versatile
- Based on approximation & data
- Computationally heavy

Linear/Logistic Regression | **coefficients**

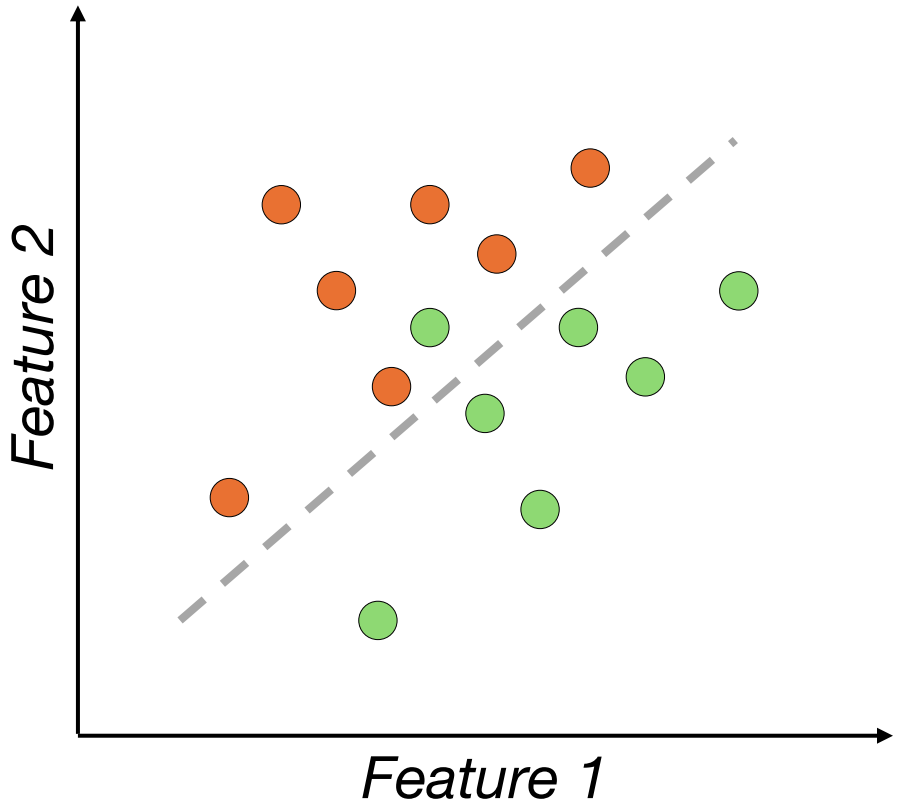
Linear combination of input features:

$$y = c_0 + c_1 x_1 + c_2 x_2 \ldots + c_n x_n$$

Straightforward interpretation:
big value → significant importance
low value → low importance

Sensetive to scaling
Stored in a fitted attribute: **.coef**

```
LinearModel.coef_[0]
```



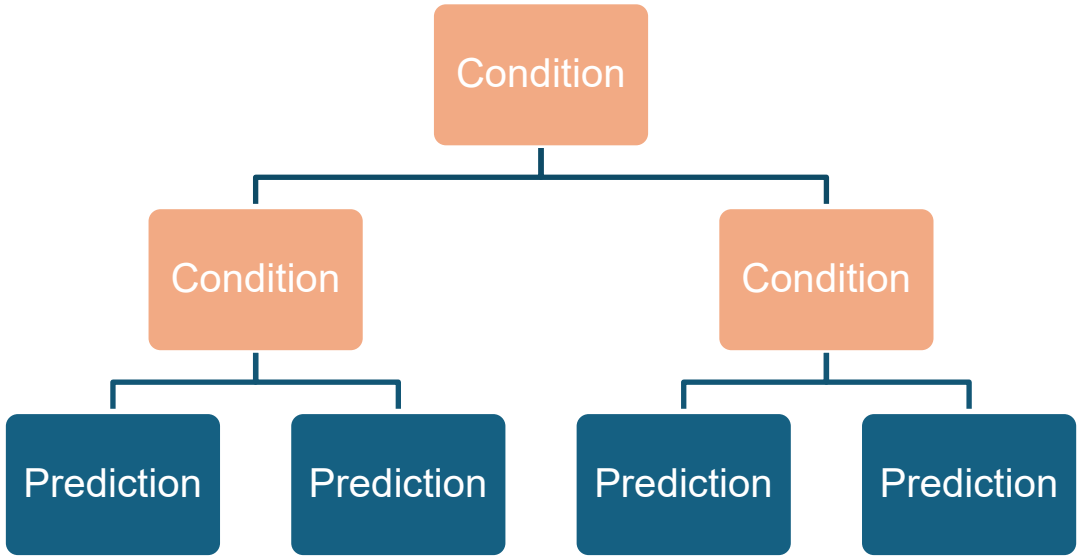
Tree based models | **Impurity importance**

Computed as the mean and standard deviation of accumulation of the impurity decrease within each tree (**Gini importance**).

- based on the **training** data
- correlated features split the credit
- high-cardinality/continuous features bias

Stored as fitted attribute: **.feature_importances_**

```
TreeBasedModel.feature_importances_[0]
```



Tree based models | **Estimators**

Exploration of individual tree structures is possible with **sklearn.tree** methods: **`export_text`** and **`export_graphviz`**.

Stored as fitted attribute: **.estimators_**

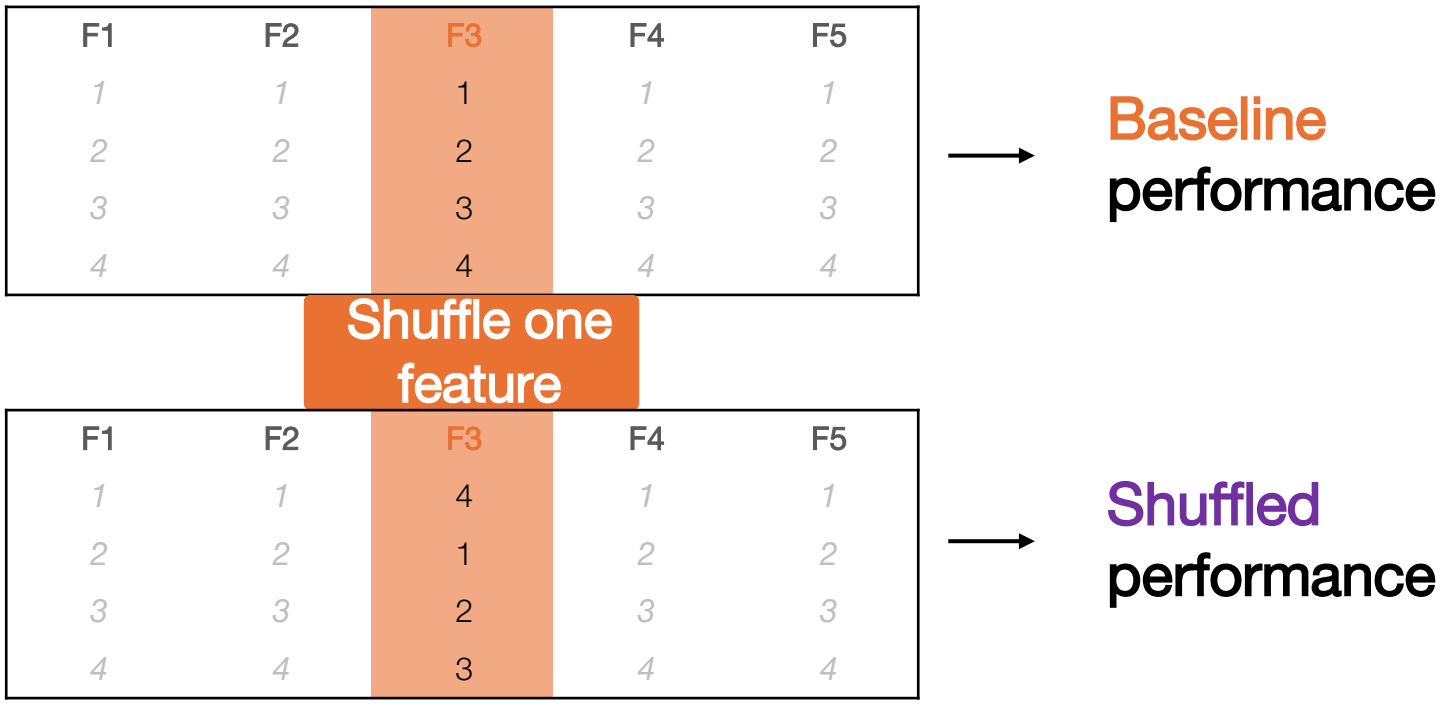
```
# X - number of individual tree
TreeBasedModel.estimators_[X]
```

```
|--- systolic_blood_pressure <= 137.00
|   |--- sex_isFemale <= 0.50
|   |   |--- platelets <= 0.50
|   |   |   |--- serum_protein <= 8.15
|   |   |   |   |--- bmi <= 18.97
|   |   |   |   |   |--- serum_albumin <= 4.15
|   |   |   |   |   |   |--- class: 1.0
|   |   |   |   |   |--- serum_albumin >  4.15
|   |   |   |   |   |   |--- serum_albumin <= 4.25
|   |   |   |   |   |   |   |--- class: 0.0
|   |   |   |   |   |   |--- serum_albumin >  4.25
|   |   |   |   |   |   |   |--- bmi <= 16.93
|   |   |   |   |   |   |   |   |--- class: 1.0
|   |   |   |   |   |   |   |--- bmi >  16.93
|   |   |   |   |   |   |   |   |--- hematocrit <= 43.50
|   |   |   |   |   |   |   |   |   |--- red_blood_cells <= 4.62
|   |   |   |   |   |   |   |   |   |   |--- class: 0.0
|   |   |   |   |   |   |   |   |   |--- red_blood_cells >  4.62
|   |   |   |   |   |   |   |   |   |   |--- class: 1.0
|   |   |   |   |   |   |   |   |--- hematocrit >  43.50
|   |   |   |   |   |   |   |   |   |--- class: 1.0
....
```

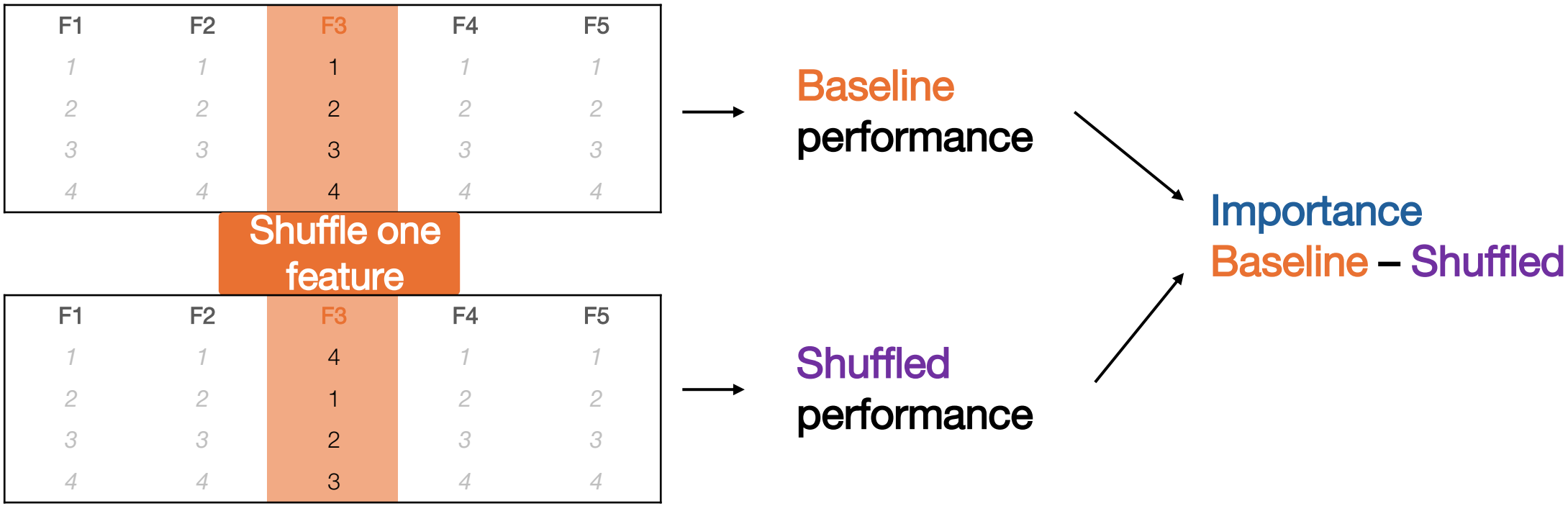
Permutation importance

- **Shuffle one feature** — keep its values, destroy their order
- If the score drops, the model depended on it
- **Model-agnostic**: any model, any metric
- **Global picture** of what the model relies on

Permutation importance | How it works



Permutation importance | How it works

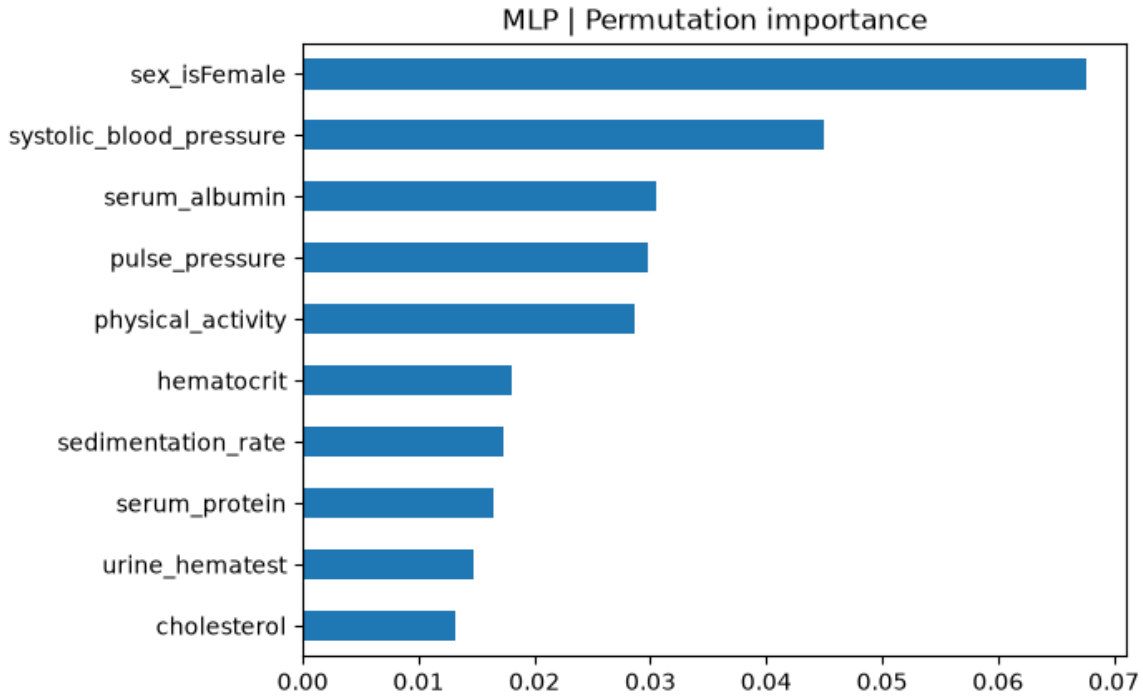


Permutation importance | How it works

```
from sklearn.inspection import permutation_importance
```

```
result= permutation_importance(
        AnyModel, X_train, y_train,
        n_repeats=10, random_state=42,
        scoring="f1")

# Access
result.importances_mean
```



Permutation importance | Conclusions

Good to know

- Model-agnostic, versatile
- Re-score only: no retraining needed
- Repeat & average (n_repeats) for stability

Watch out

- Correlated features can hide each other
- Cost grows with features × repeats
- Global summary — no single-prediction (local) story

SHAP | **SH**apley **A**dditive ex**P**lanations

- **Explains one prediction** — feature by feature
- Uses **shapely values** from game theory
- **Local method** — aggregate over instances for a **global view**
- **Model-agnostic** — works for any model family

Every prediction = baseline + the sum of its SHAP values

SHAP | The game behind SHAP

Player → a gene in the pathway (A, B, C)

Coalition S → any subset of genes

Value v(S) → what that subset produces

Marginal contribution → how much v(S) changes when a gene joins

Shapley value → the gene's fair share of the output

gene A · activator

gene B · activator

gene C · inhibitor

SHAP | Three genes, one pathway

Active genes	Output
∅ — none	0
A	100
B	40
C	0
A + B	150
A + C	90
B + C	30
A + B + C	140

A + B reach 150 — they **boost** each other

C always costs the team: 100 → 90, 40 → 30, 150 → 140

How to split the final 140 fairly?

SHAP | Averaging over all join orders

Active genes	Output
∅ — none	0
A	100
B	40
C	0
A + B	150
A + C	90
B + C	30
A + B + C	**140**

A's contribution

Join order	Before A	A joins	A's contribution
A → B → C	∅ = 0	{A} = 100	**+100**
A → C → B	∅ = 0	{A} = 100	**+100**
B → A → C	{B} = 40	{A,B} = 150	**+110**
B → C → A	{B,C} = 30	{A,B,C} = 140	**+110**
C → A → B	{C} = 0	{A,C} = 90	**+90**
C → B → A	{C,B} = 30	{A,B,C} = 40	**+110**

Same gene, different credit
it depends on who is already present.

SHAP | Three genes, one pathway

A's contribution

Before	A joins	Cases	A's contribution
∅	{A}	2	**+100**
{B}	{A,B}	1	**+110**
{C}	{A,C}	1	**+90**
{B, C}	{A,B,C}	2	**+110**

$\phi A = (2\times100+2\times110+90+110) / 6 = 103.33$

Join order	Before A	A joins	A's contribution
A → B → C	∅ = 0	{A} = 100	**+100**
A → C → B	∅ = 0	{A} = 100	**+100**
B → A → C	{B} = 40	{A,B} = 150	**+110**
B → C → A	{B,C} = 30	{A,B,C} = 140	**+110**
C → A → B	{C} = 0	{A,C} = 90	**+90**
C → B → A	{C,B} = 30	{A,B,C} = 40	**+110**

SHAP | Averaging over all join orders

Before	A joins	Cases	A's contribution
∅	{A}	2	+100
{B}	{A,B}	1	+110
{C}	{A,C}	1	+90
{B, C}	{A,B,C}	2	+110

$\phi A = (2\times100+2\times110+90+110) / 6 = 103.33$

*Repeat for **B**, repeat for **C**…*

$\phi B = (2\times40 + 50 + 30 + 2\times50) / 6 = 43.33$

$\phi C = (2\times0 + (-10) + (-10) + 2\times(-10)) / 6 = -6.67$

A · 103.33

B · 43.33

C · –6.67

103.33 + 43.33 – 6.67 = 140

Total stays the same: nothing left unassigned, negative shares allowed

SHAP | The Shapley value formula

Here is the famous Shapley formula:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\ (|N| - |S| - 1)!}{|N|!} \left[v(S \cup \{i\}) - v(S) \right]$$

ϕA = 2/6×100 + 1/6×110 + 1/6×90 + 2/6×110 = 103.33

For every possible team S, measure what of interest adds, then average those additions with the right weights.

SHAP | Why exact SHAP is expensive

Advantages

- Principled, fair attribution: game-theory axioms.
- Local to global.
- Symmetrical.

- Exact Shapley = evaluate every feature subset — grows exponentially
- In practice: approximate/subset

Features M	Coalitions 2^M
10	1,024
20	1,048,576
30	1,073,741,824

M = number of features

SHAP | New API

Old way: direct experts, tedious

```
RF_explainer = shap.TreeExplainer(RF_clf)
RF_sv = RF_explainer.shap_values(X_test)

shap.summary_plot(shap_values[:,:,1], X_test,
plot_type="bar")
```

- You pick the explainer: **TreeExplainer**
- **.shap_values()** on data → raw NumPy arrays
- Arrays → your own plotting code / different logic for shap plots

New way — one object does it

```
explainer = shap.Explainer(TreeModel)
# auto => algorithm="tree"
RF_sv = explainer(X_test) # => can plot
without other objects

shap.plots.bar(RF_sv[:,:,1])
```

- **shap.Explainer** picks the algorithm / you specify
- **Explanation** object → shap.plots.* directly
- Explanation object is convenience. Contains:
- **.values**, **.base_values**, **.data**, **.feature_names**

SHAP | New API

Old way: direct experts, tedious

```
RF_explainer = shap.TreeExplainer(RF_clf)
RF_sv = RF_explainer.shap_values(X_test)

shap.summary_plot(shap_values[:,:,1], X_test,
plot_type="bar")
```

- You pick the explainer: **TreeExplainer**
- **.shap_values()** on data → raw NumPy arrays
- Arrays → your own plotting code / different logic for shap plots

New way — one object does it

```
explainer = shap.Explainer(TreeModel)
# auto => algorithm="tree"
RF_sv = explainer(X_test) # => can plot
without other objects

shap.plots.bar(RF_sv[:,:,1])
```

- **shap.Explainer** picks the algorithm / you specify
- **Explanation** object → shap.plots.* directly
- Explanation object is convenience. Contains:
- **.values**, **.base_values**, **.data**, **.feature_names**

SHAP | New API |Coalitions vs orderings

Kernel SHAP — coalitions

Sample a subset S — then watch what one more feature adds.

S: x1 x2 x3 x4 x5

S ∪ {x5}: x1 x2 x3 x4 **x5**

$\Delta = v(S \cup \{x5\}) - v(S)$

more subsets: {x2}, {x1, x2, x4}, … — each measured the same way

Permutation SHAP — orderings

March left to right — each feature joins the growing prefix.

x3 → x1 → **x4** → x2 → x5

x4 joins {x3, x1} → its Δ

x2 → x5 → x1 → x3 → x4

more orderings: x1 → x5 → …, x4 → x2 → …

Two ways to sample the same game — both land on the same Shapley values.

SHAP | New API |Coalitions vs orderings

2 genes: **A** and **B**

A 10 | **B** 20 | **A**,**B** 50

Kernel SHAP — coalitions

A joins {} => +10
A joins {**B**} => +30

Weights **1/2,** so **ϕA** 40/2 = **20**

Permutation SHAP — orderings

A then **B** => +10(**A**) then + 40(**B**)
B then **A** => +20(**B**) then +30(**A**)

ϕA = average(10+30) = **20**

Background: clustered centroids

"SHAP values by definition"

Background: actual data

Much faster in practice than Kernel SHAP

Two ways to sample the same game — both land on the same Shapley values.

SHAP | Random Forest | TreeExplainer

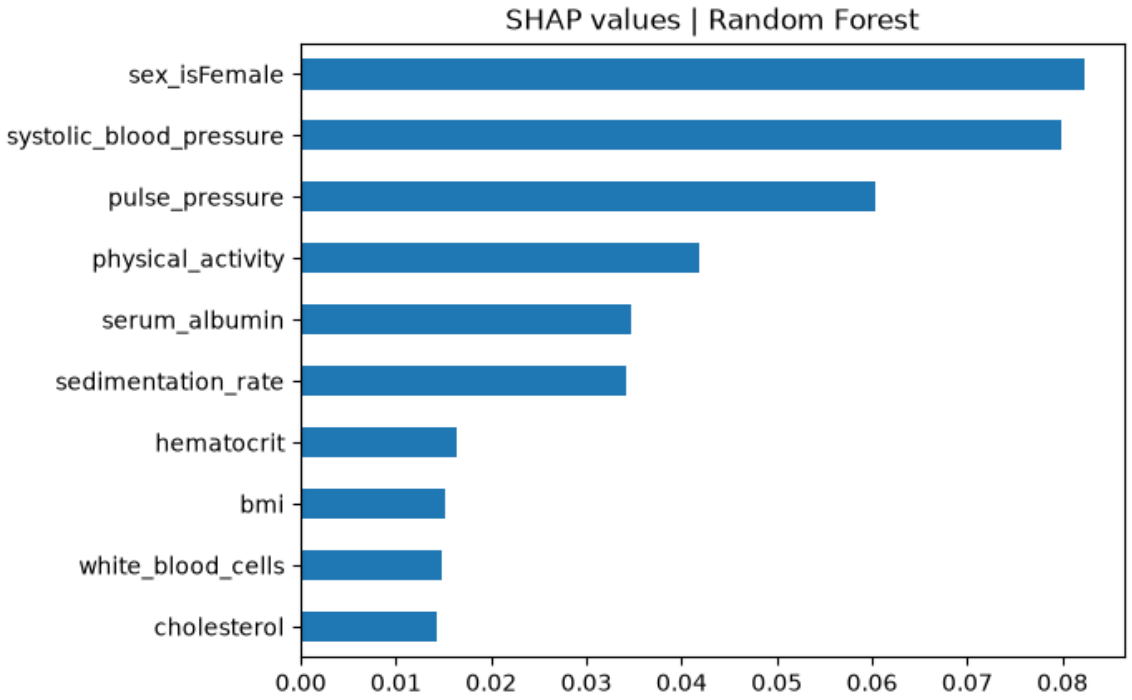
Exact tree math replaces sampling

```
X_test_small = X_test.iloc[:200, :]

RF_explainer = shap.Explainer(RF_clf)
# algorithm="tree"
RF_sv = RF_explainer(X_test_small)
RF_sv_values = RF_sv.values

RF_mean_sv = np.abs(
    RF_sv_values[:,:,1]).mean(axis=0)

plot_features_pd(
    RF_mean_sv, X.columns,
    "SHAP values | Random Forest")
```



Explanation | mean |SHAP| over 200 test samples.

SHAP | MLP | KernelExplainer

No internals to read, so SHAP samples coalitions

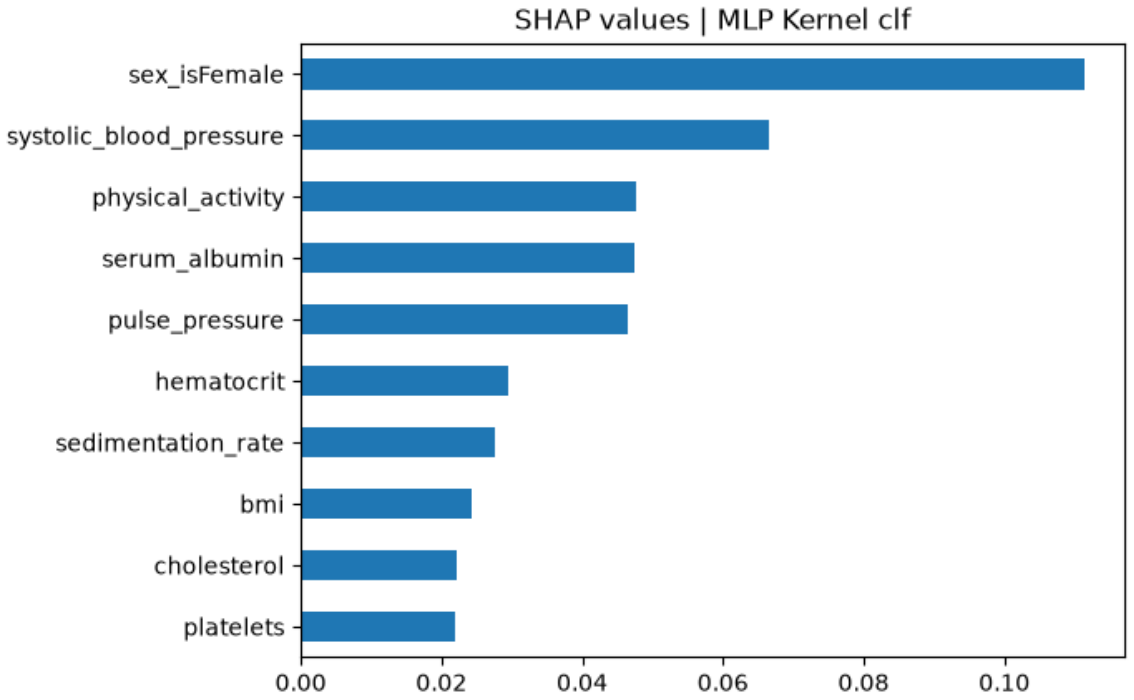
```
bg = shap.kmeans(X_train_scaled, 10)

MLP_kernel_explainer = shap.KernelExplainer(
    MLP_clf.predict_proba, bg)

MLP_sv_only_kernel = (
    MLP_kernel_explainer.shap_values(
        X_test_scaled)[:,:,1])

MLP_sv_only_mean_kernel = np.abs(
    MLP_sv_only_kernel).mean(axis=0)

plot_features_pd(
    MLP_sv_only_mean_kernel, X.columns,
    "SHAP values | MLP Kernel clf")
```



Explanation | k-means background (10 centroids); mean |SHAP| across the test set.

SHAP | MLP | PermutationExplainer

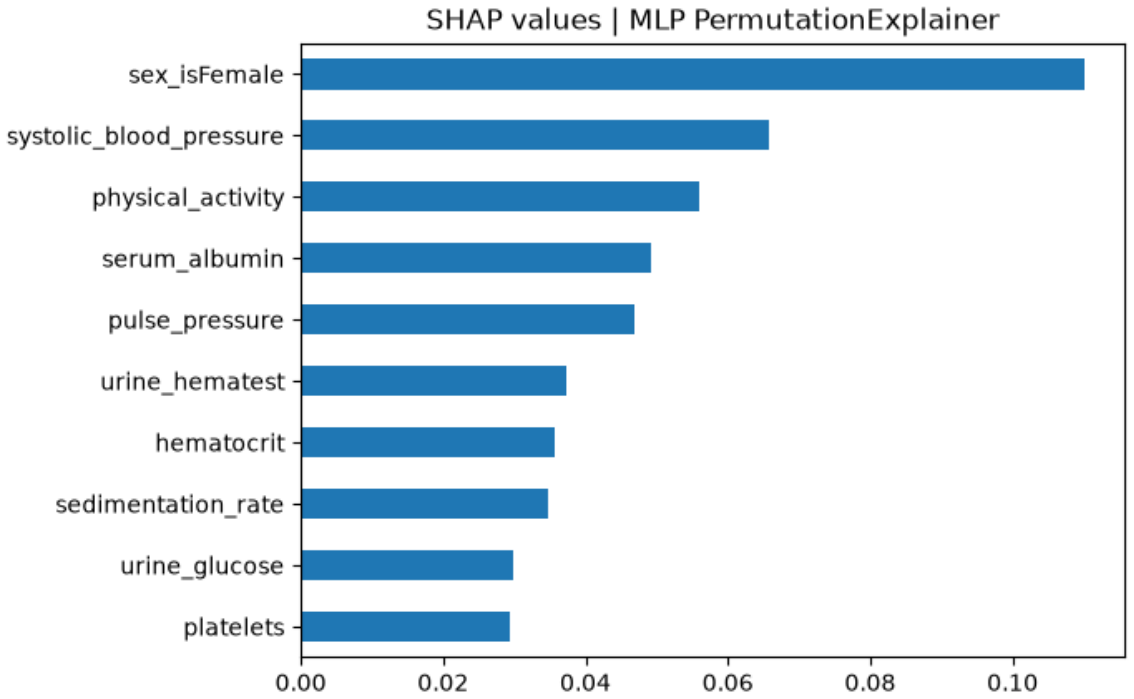
The modern replacement for kernel SHAP: faster, and the ranking barely moves

```
bg = shap.sample(
    X_train_scaled, 100, random_state=42)

MLP_explainer = shap.Explainer(
    MLP_clf.predict_proba, bg, seed=42)

MLP_sv_scaled = MLP_explainer(X_test_scaled)
MLP_sv_scaled_only = MLP_sv_scaled.values[:,:,1]
MLP_sv_scaled_mean = np.abs(
    MLP_sv_scaled_only).mean(axis=0)

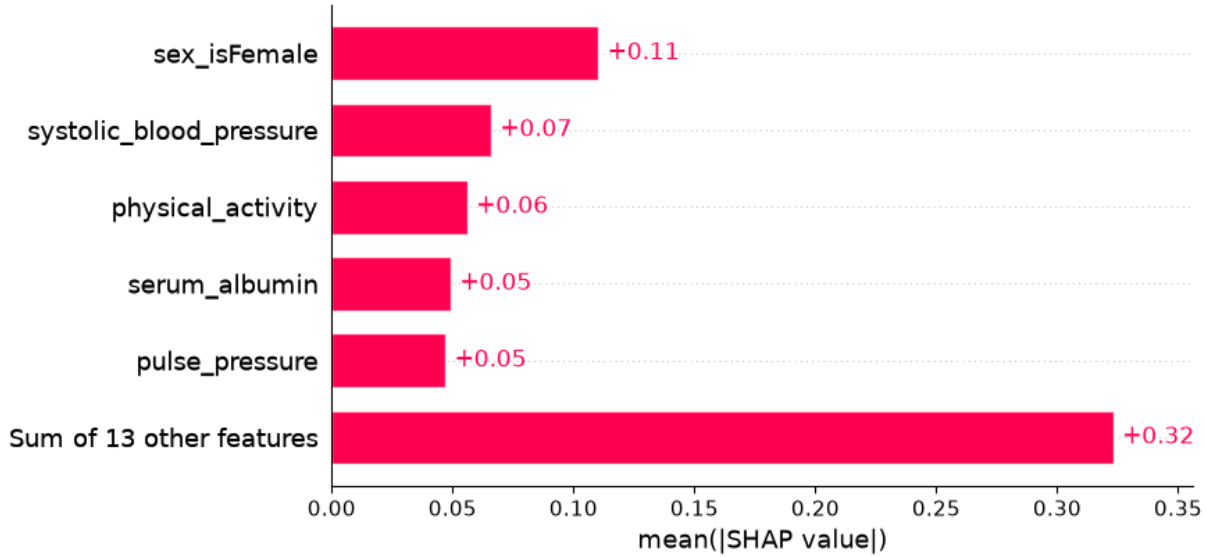
plot_features_pd(
    MLP_sv_scaled_mean, X.columns,
    "SHAP values | MLP PermutationExplainer")
```



Explanation | 100-sample background; mean |SHAP| across the test set.

SHAP | MLP | Bar Plot

```
# mean(|SHAP value|) per feature
shap.plots.bar(MLP_sv_scaled[:,:,1], max_display=6)
```

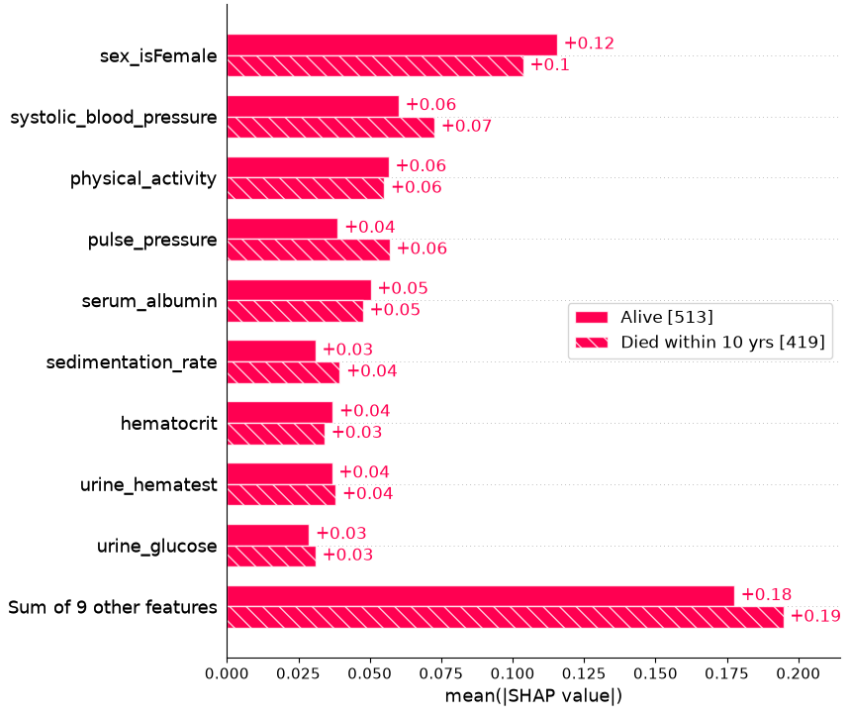


Explanation | each bar = average |push| on the target (Died in 10 years) class.

SHAP | MLP | Cohorts => |SHAP| split

```
classes = [
    "Alive" if cls == 0 else "Died within 10 yrs"
    for cls in y_test]

shap.plots.bar(
    MLP_sv[:,:,1].cohorts(classes).abs.mean(0))
```

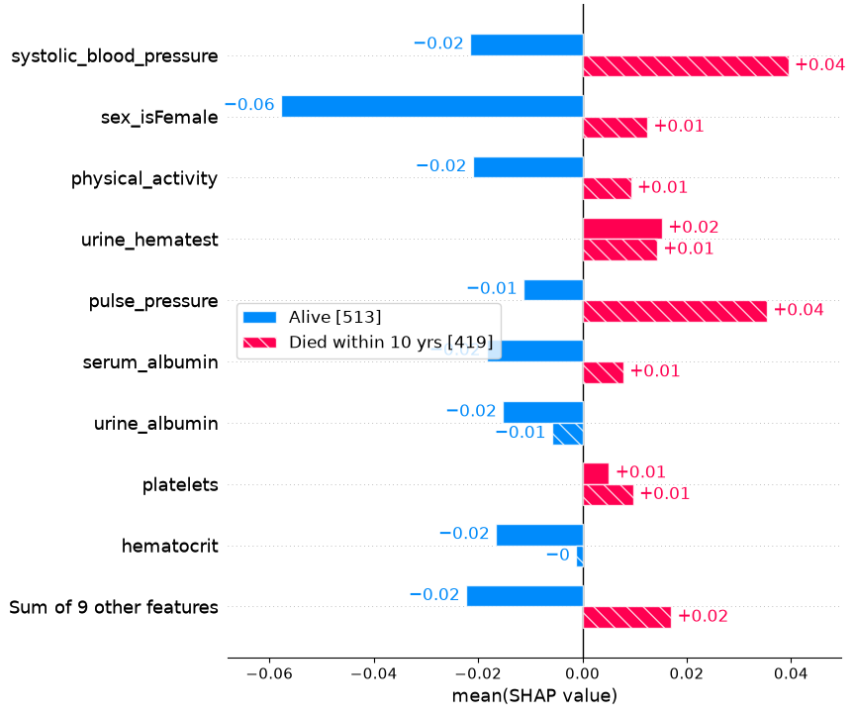


Explanation | mean |push| within each group for the feature. Abs makes all features to focus on “how strong” feature pushes the prediction.

SHAP | MLP | Cohorts => signed means

```
classes = [
    "Alive" if cls == 0 else "Died within 10 yrs"
    for cls in y_test]

shap.plots.bar(
    MLP_sv[:,:,1].cohorts(classes).mean(0))
```

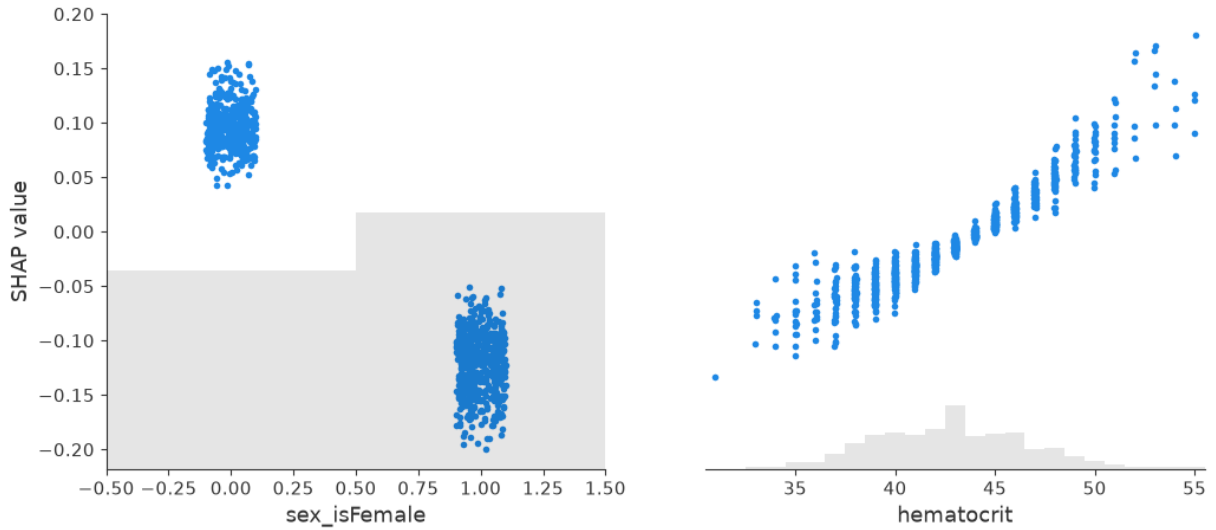


Explanation | mean push within each group for the feature.
No abs() => fair representation across cohorts.

SHAP | MLP | Scatter

```
MLP_sv = copy(MLP_sv_scaled)
MLP_sv.data = scaler.inverse_transform(
    MLP_sv_scaled.data)

shap.plots.scatter(
    MLP_sv[:, ["sex_isFemale", "hematocrit"], 1])
```

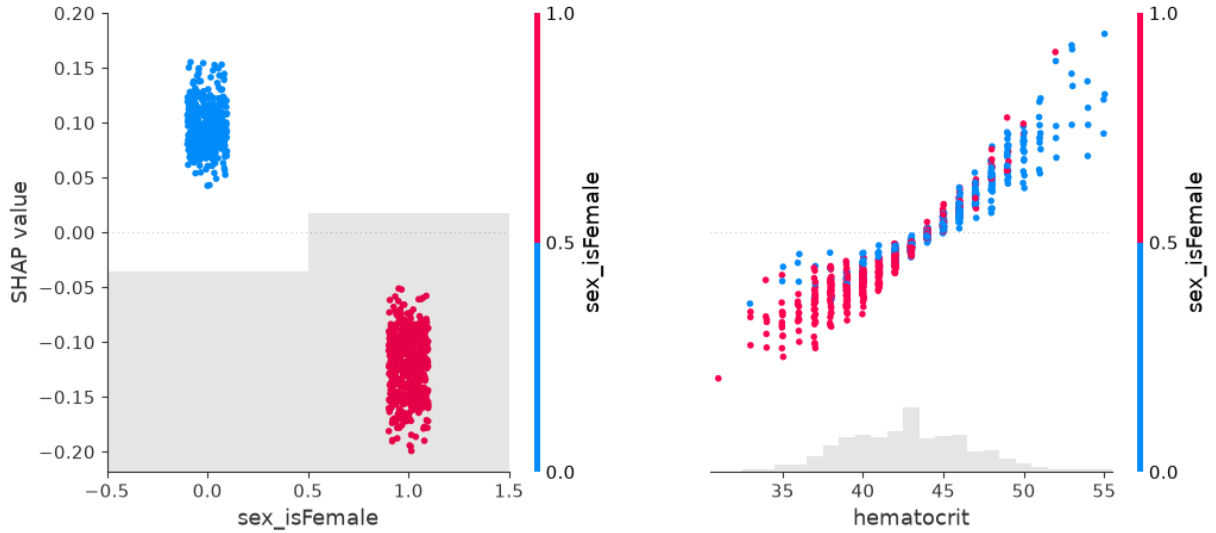


Explanation | x = feature value, y = that patient's SHAP push; grey histogram = samples distribution.

SHAP | MLP | Scatter: feature colour

Colour overlay by another feature

```
shap.plots.scatter(
    MLP_sv[:, ["sex_isFemale", "hematocrit"], 1],
    color=MLP_sv[:, "sex_isFemale", 1])
```

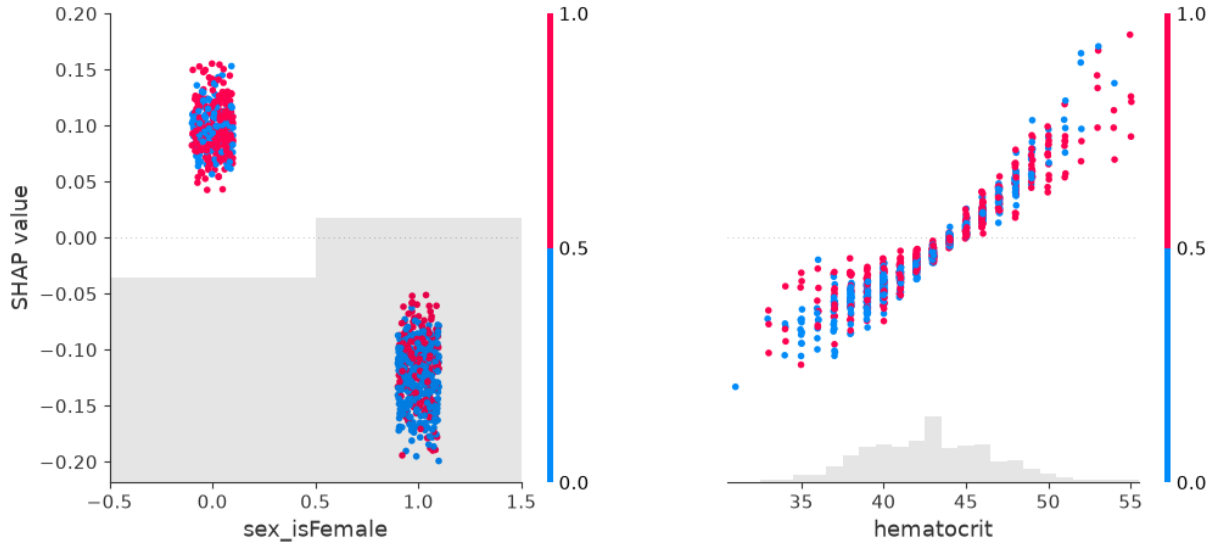


Explanation | x = feature value, y = that patient's SHAP push; grey histogram = samples distribution. Colour: another feature mask.

SHAP | MLP | Scatter: outcome colour

Colour overlay by target class

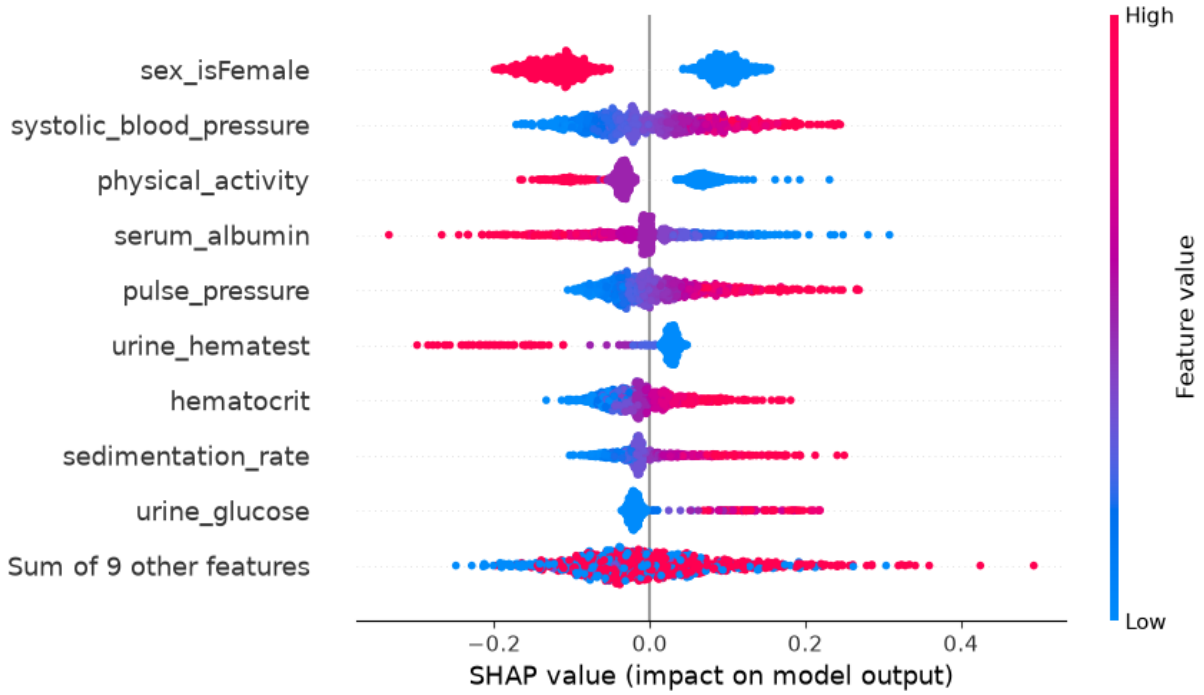
```
shap.plots.scatter(
    MLP_sv[:, ["sex_isFemale", "hematocrit"], 1],
    color=y_test.values)
```



Explanation | x = feature value, y = that patient's SHAP push; grey histogram = samples distribution. Colour: target class mask

SHAP | MLP | Beeswarm

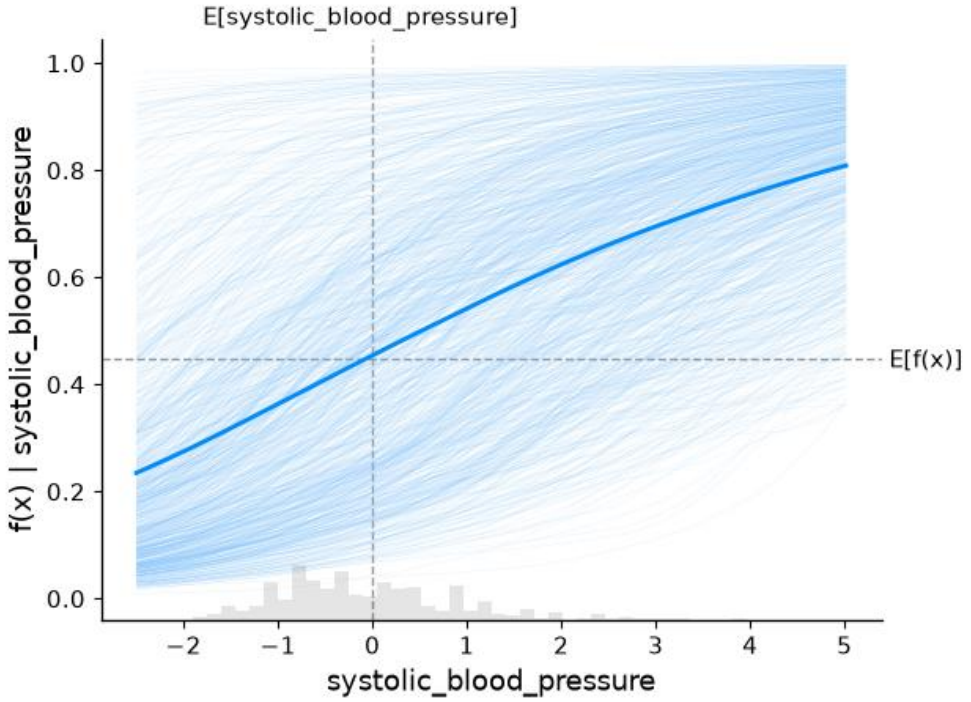
```
shap.plots.beeswarm(MLP_sv[:,:,1])
```



Explanation | one dot = patient × feature; x = push (right = toward "Died"); colour = the feature's value.

SHAP | MLP | Partial Dependence

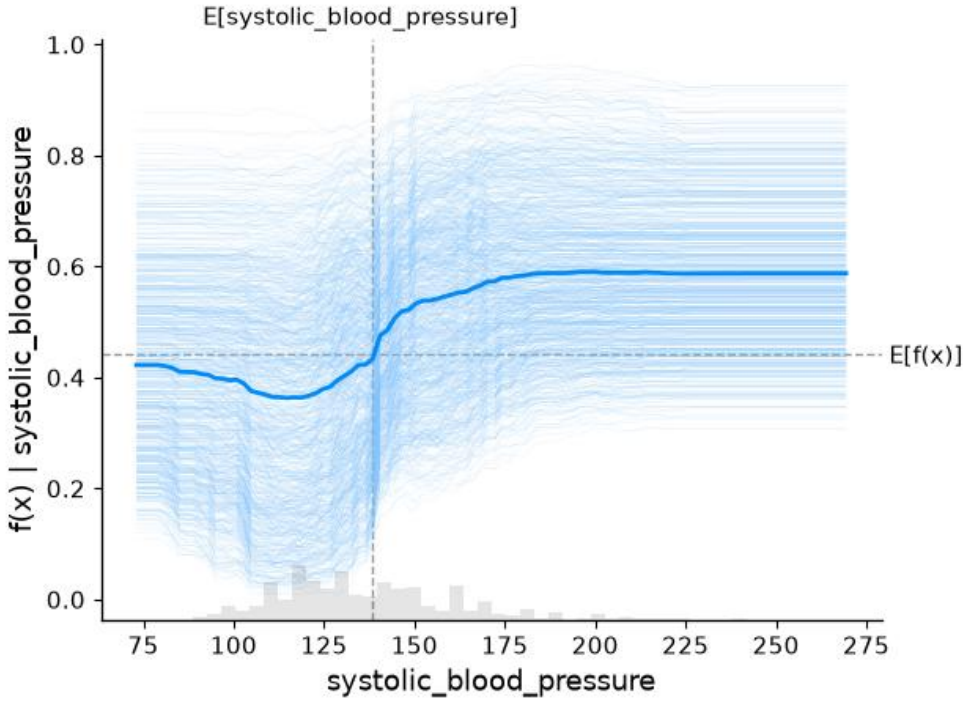
```
shap.plots.partial_dependence(
    "systolic_blood_pressure",
    lambda data: MLP_clf.predict_proba(data) [:,1],
    X_test_scaled, ice=True,
    model_expected_value=True,
    feature_expected_value=True)
```



Explanation | thick line = average prediction with every patient's pressure set to that value (scaled units); thin lines = individual patients (ICE); dashed = baseline 0.477.

SHAP | RF | Partial Dependence

```
shap.plots.partial_dependence(
    "systolic_blood_pressure",
    lambda data: RF_clf.predict_proba(data) [:,1],
    X_test, ice=True,
    model_expected_value=True,
    feature_expected_value=True)
```



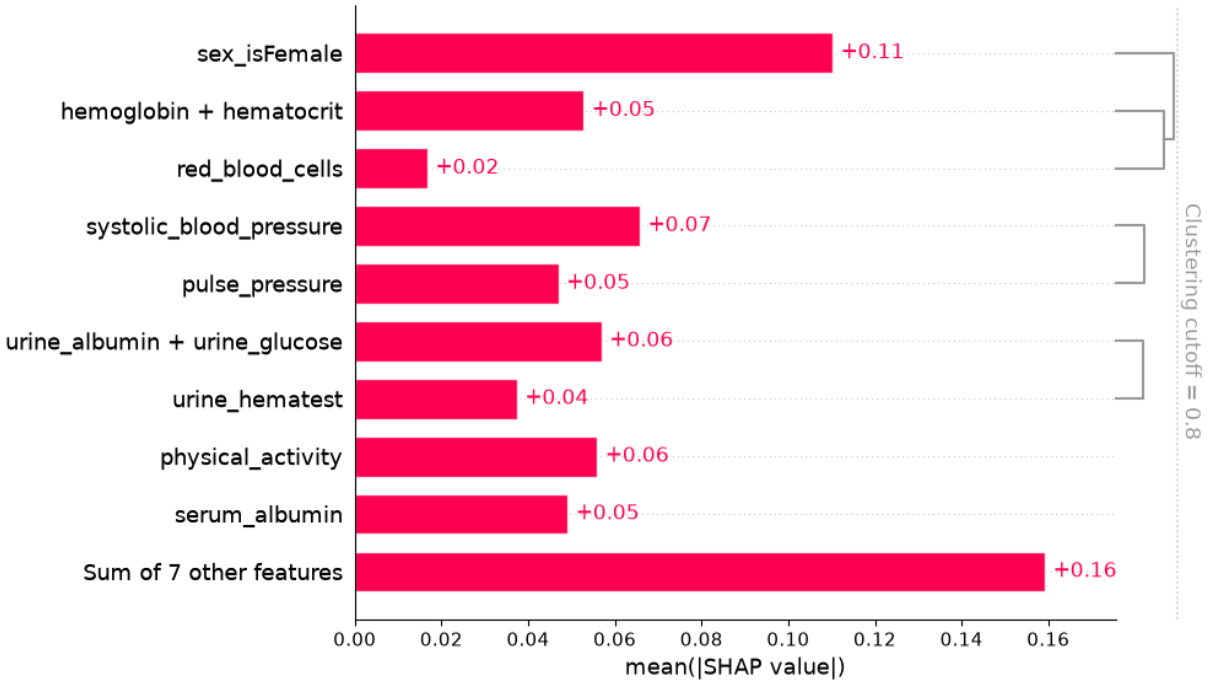
Explanation | the same question in raw mmHg; flat below ~135, then a jump — trees split space, they don't draw curves.

SHAP | MLP | Clustering Bar

Substitutes share credit

```
MLP_clustering = shap.utils.hclust(X_test, y_test)

shap.plots.bar(
    MLP_sv[:,:,1], clustering=MLP_clustering,
    clustering_cutoff=0.8, max_display=10)
```

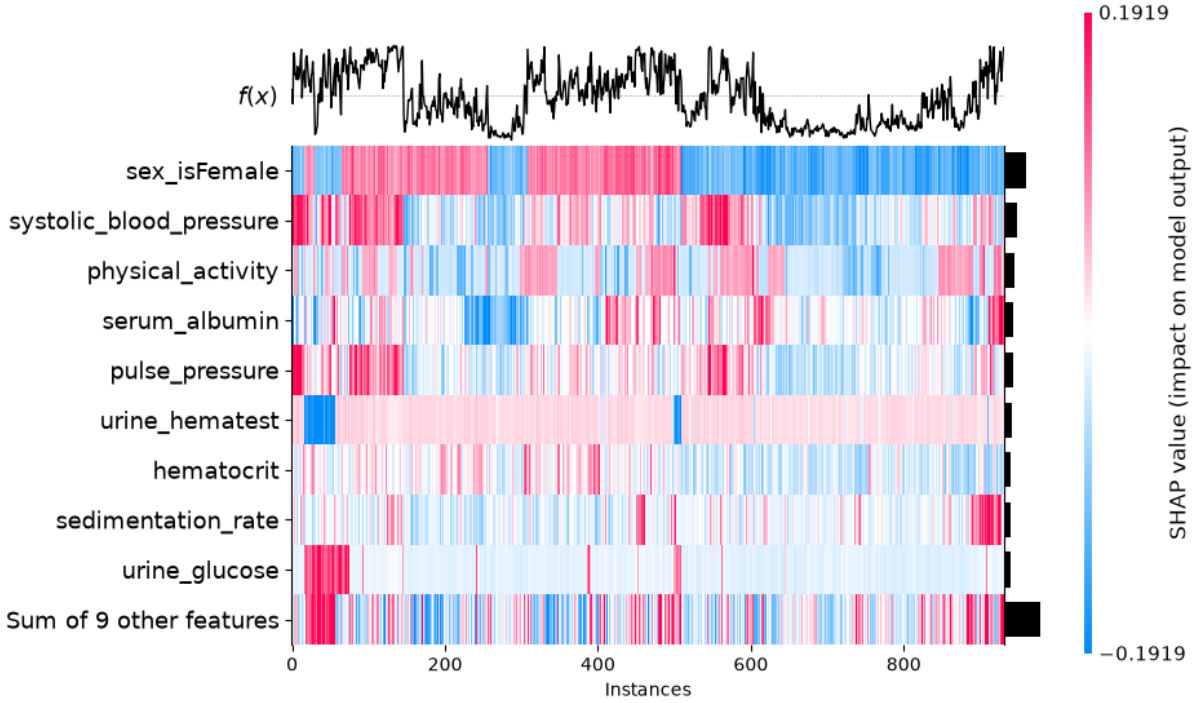


Explanation | dendrogram groups features whose SHAP patterns move alike (cutoff 0.8); merged bars = combined push, correlation-aware.

SHAP | MLP | Heatmap: full test set

Patients reordered so similar explanations sit together

```
shap.plots.heatmap(MLP_sv[:,:,1])
```

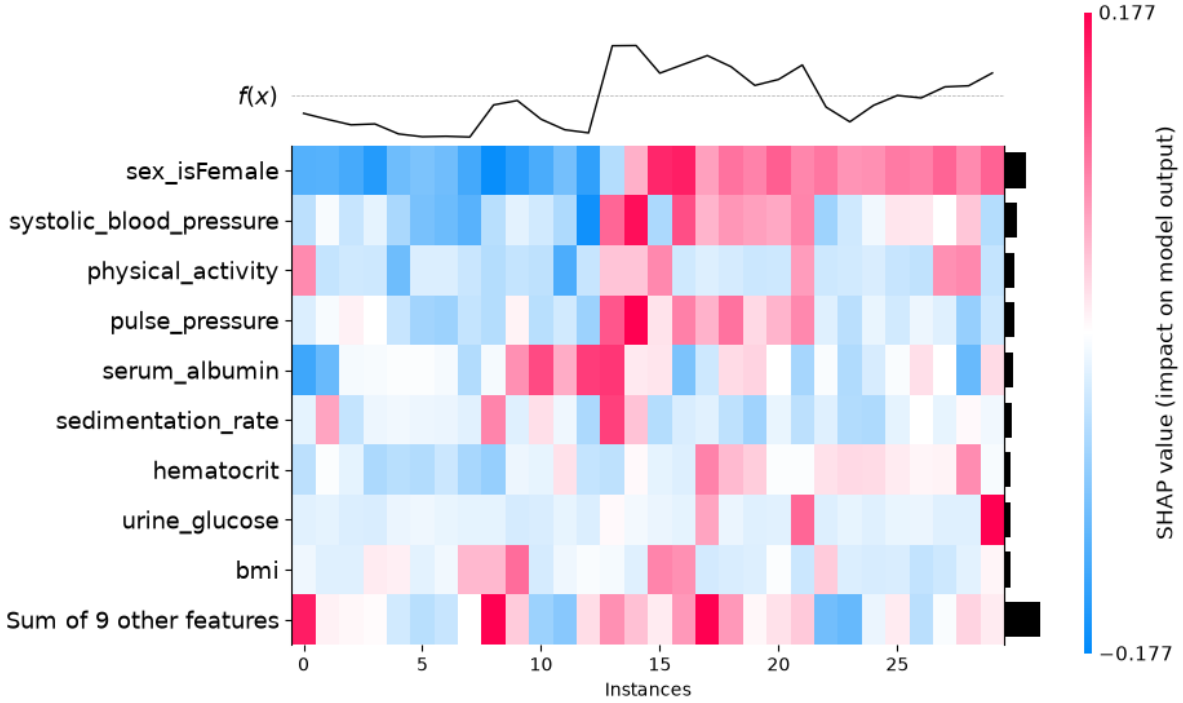


Explanation | column = one patient, cell = one push (red up / blue down); black curve on top = each patient's f(x), dashed = baseline; coherent colour blocks = similar patients.

SHAP | MLP | Heatmap: slice

New slice => new order

```
shap.plots.heatmap(MLP_sv[:30,:,1])
# any number of samples (2+); order != data order
```



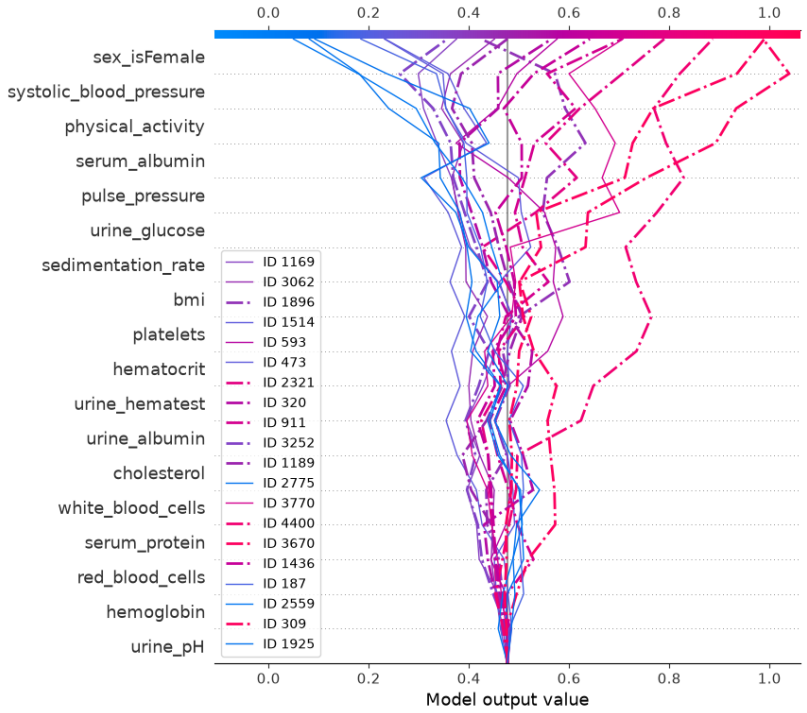
Explanation | the same plot for the first 30 test patients (not grouped by outcome).

SHAP | MLP | Decision Plot

Each line walks one patient from the baseline to their predicted risk

```
# sub, died_idx: 10 Alive + 10 Died patients
base_value = MLP_sv.base_values[0,1]
shap_values = MLP_sv.values[sub,:,1]
feature_names = MLP_sv.feature_names
highlight = np.isin(sub, died_idx)
legend_labels = [f"ID {idx}"
                 for idx in X_test.index[sub]]

shap.plots.decision(
    base_value, shap_values,
    feature_names=feature_names,
    highlight=highlight,
    legend_labels=legend_labels)
```



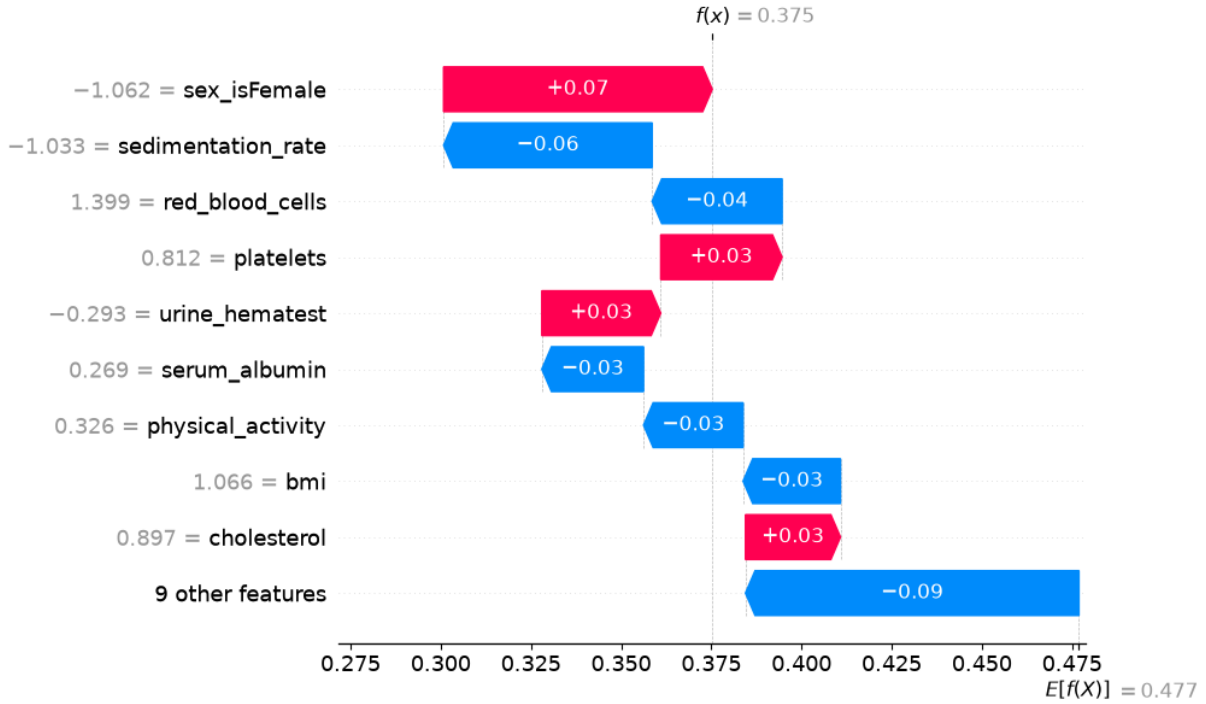
Explanation | 10 survivors + 10 deaths; lines start at E[f] ≈ 0.477 and end at the prediction; dash-dot = actually died; left endings = net pushed down.

SHAP | MLP | Waterfall: survivor

```
X_test_scaled_sample = X_test_scaled.iloc[[0], :]

sv_test_sample = MLP_explainer(
    X_test_scaled_sample, max_evals=2000)

shap.plots.waterfall(sv_test_sample[0,:,1])
```



Explanation | one survivor (ID 1169); bars = each feature's push (red toward "Died", blue away); f(x) = 0.375 < 0.5 → predicted to survive.

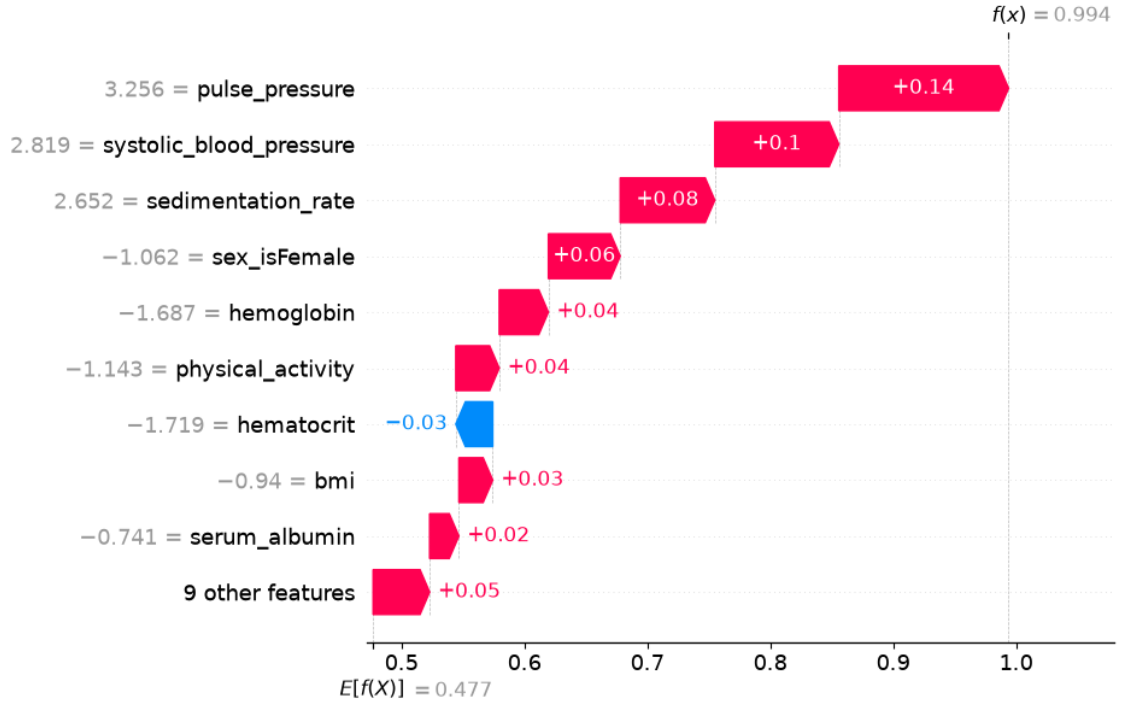
SHAP | MLP | Waterfall: highest risk

```
idx_risk = np.argmax(
    MLP_clf.predict_proba(X_test_scaled)[:,1])

X_test_scaled_risk = (
    X_test_scaled.iloc[[idx_risk], :])

sv_risk_sample = MLP_explainer(
    X_test_scaled_risk, max_evals=2000)

shap.plots.waterfall(sv_risk_sample[0,:,1])
```



Explanation | the test-set patient with the top predicted risk.

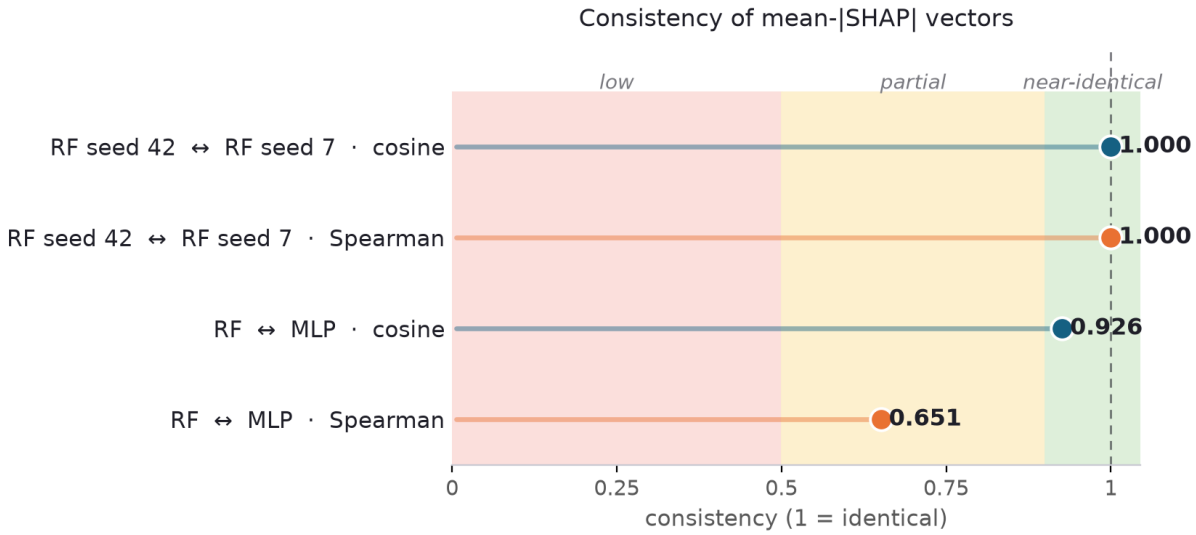
SHAP | Consistency | RF vs MLP

New seed keeps the story; a new model keeps the picture, not the order.

```python
from sklearn.metrics.pairwise import (
    cosine_similarity)
from scipy.stats import spearmanr

def explanation_consistency(vals1, vals2):
    cosine = cosine_similarity(
        [vals1], [vals2])[0,0]
    spearman = spearmanr(vals1, vals2)[0]
    print(f"Cosine   : {cosine:.3f}")
    print(f"Spearman : {spearman:.3f}")

# mean-|SHAP| vectors: RF, RF seed 7, MLP
explanation_consistency(
    RF_sv_values_mean, RF_B_sv_values_mean)
explanation_consistency(
    RF_sv_values_mean, MLP_sv_mean)
```



Explanation | each dot scores one comparison on the 0–1 scale; cosine (credit proportions) stays high; Spearman (feature ranking) drops first — the stricter test.

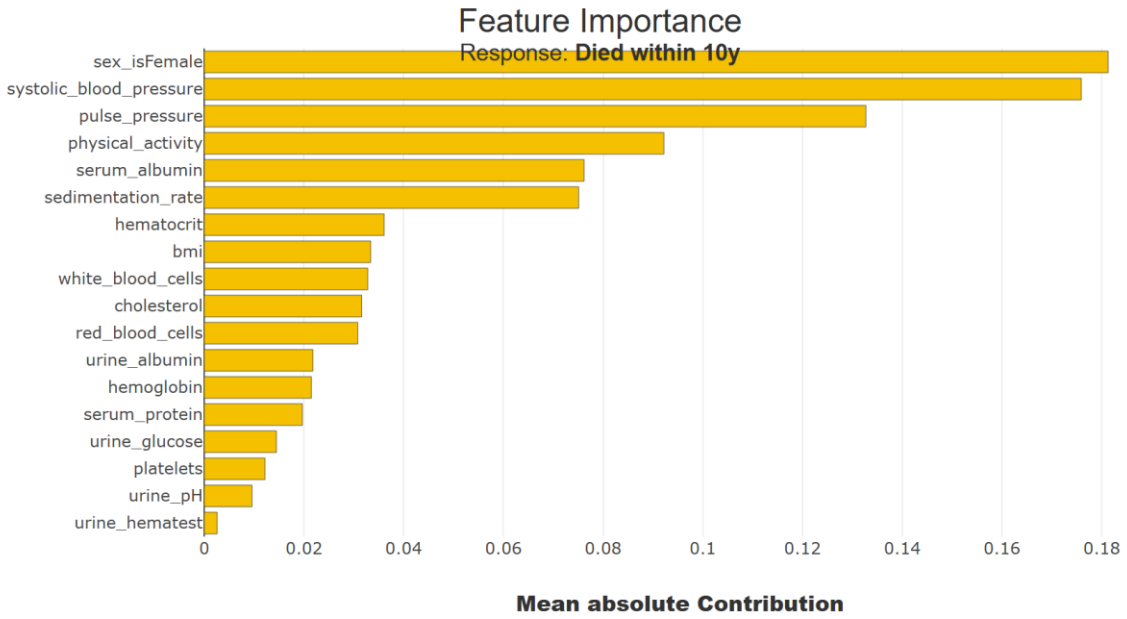
shapash | RF | Feature Importance

One compiled object — the ranking now ships with the library.

```
from shapash import SmartExplainer

RF_xpl = SmartExplainer(
    model=RF_clf,
    label_dict={0: "Alive", 1: "Died within 10y"})
RF_xpl.compile(
    x=X_test_small, y_target=y_test_small)

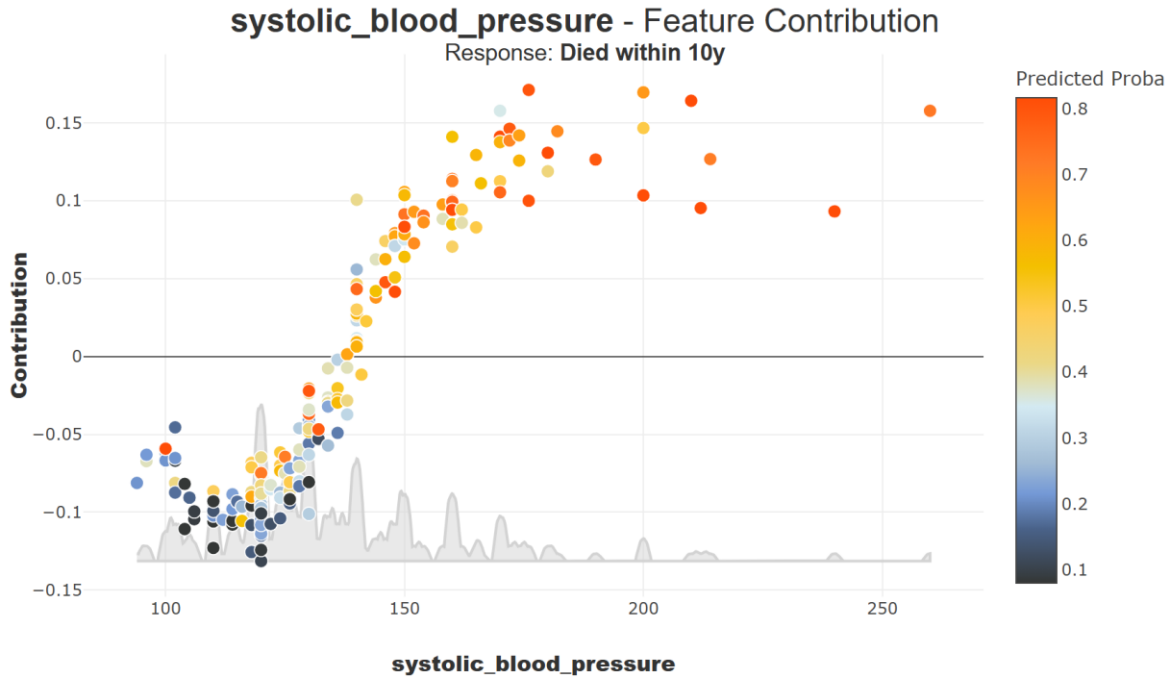
RF_xpl.plot.features_importance()
```



Explanation | Feature importance by shapash.

shapash | RF | Contribution Plot

```
# per-patient detail for one feature
RF_xpl.plot.contribution_plot(
    "systolic_blood_pressure")
```

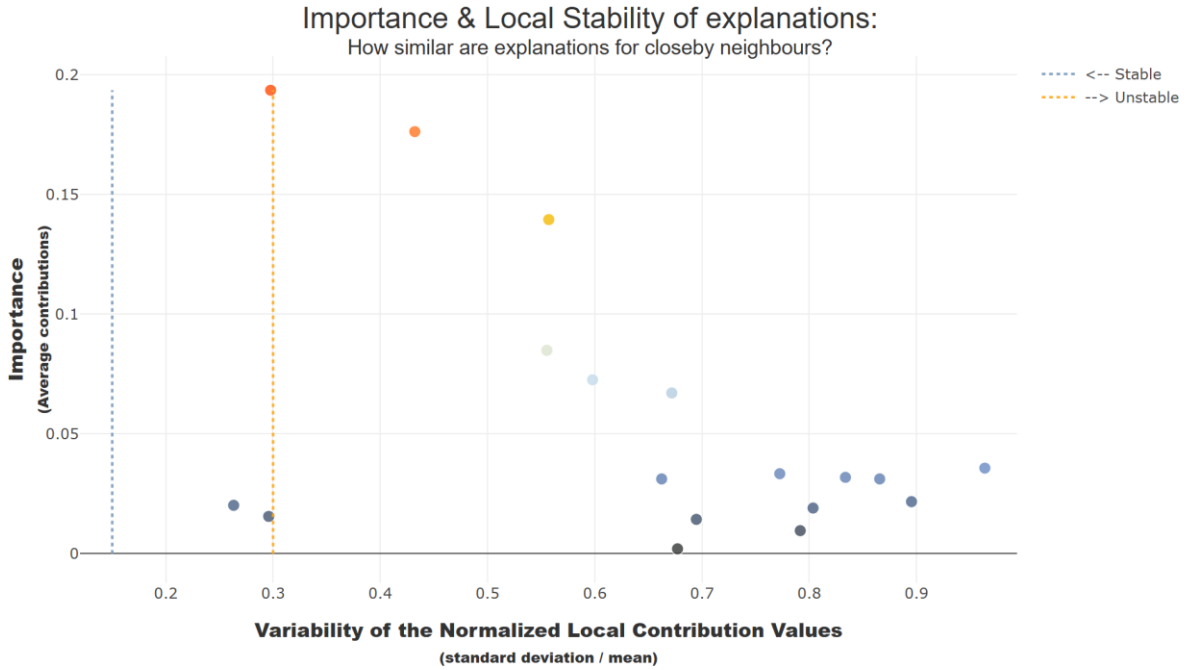


Explanation | advanced partial contribution by shapash.

shapash | RF | Stability

Importance against variability.

```
# importance vs variability, per feature
RF_xpl.plot.stability_plot()
```

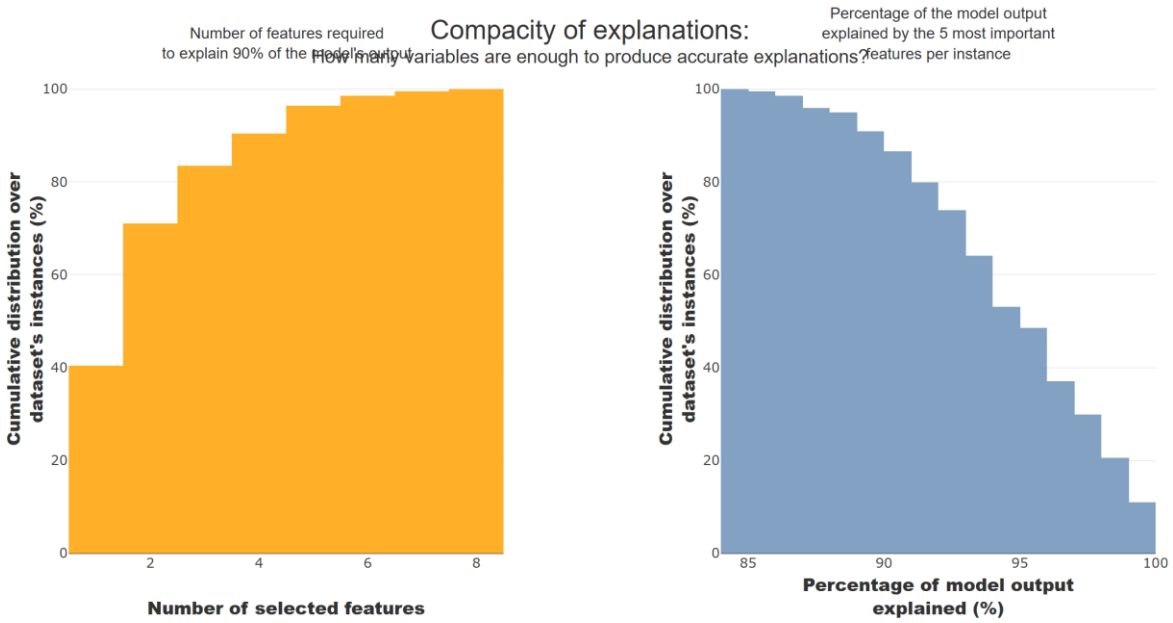


Explanation | x = how much a feature's contribution varies between similar patients, y = its average importance; left of the blue line = stable, right of orange = volatile.

shapash | RF | Compacity

How few features already reproduce the model's output?

```
# cumulative picture: how few features explain it
RF_xpl.plot.compacity_plot()
```



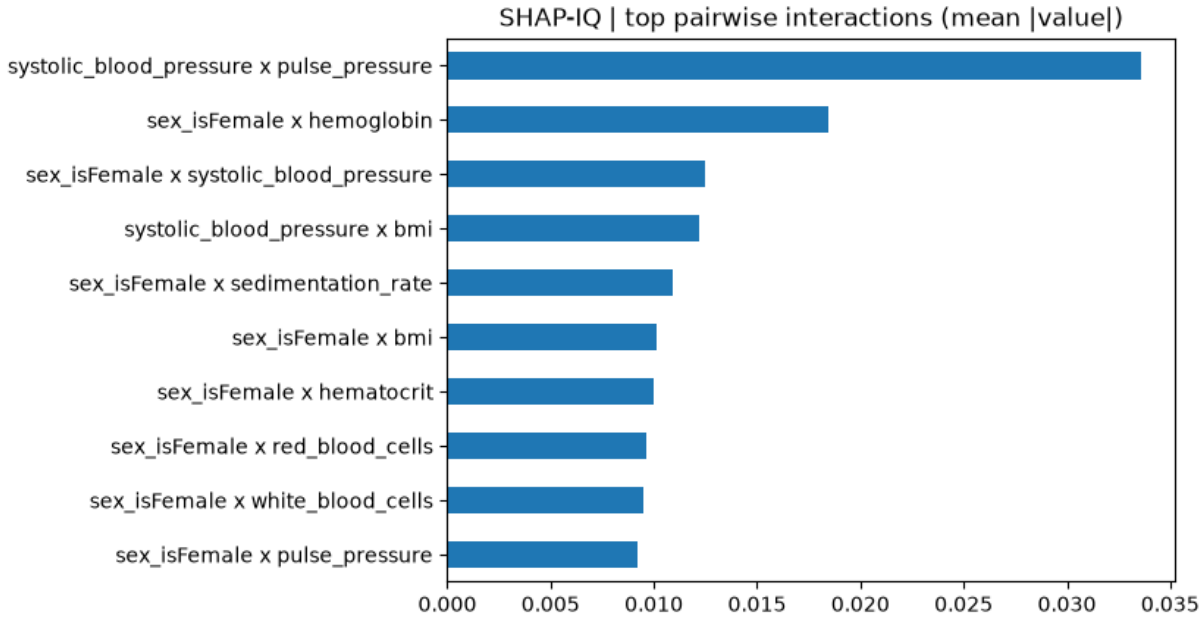
Explanation | left: share of patients explained as features are added (≈90% with 4); right: what the top-5 features reproduce (≥85% for every patient) — a compact model.

shapiq | RF | Interactions

Some credit only appears in pairs: who moves the prediction together?

```
RF_sq_explainer = shapiq.Explainer(
    model=RF_clf, index="k-SII", max_order=2,
    class_index=1)
RF_sq_values = RF_sq_explainer.explain_X(
    X_test_small.to_numpy())

RF_pair_sum = {}
for iv in RF_sq_values:
    for k, v in iv.dict_values.items():
        if len(k) == 2:
            RF_pair_sum[k] = RF_pair_sum.get(
                k, 0.0) + abs(v)
RF_pair_imp = pd.Series({
    f"{X.columns[a]} x {X.columns[b]}":
        s / len(RF_sq_values)
    for (a, b), s in RF_pair_sum.items()})
plot_features_pd(
    RF_pair_imp.values, RF_pair_imp.index,
    "SHAP-IQ | top pairwise interactions")
```



Explanation | bar = mean |interaction value| per feature pair over the same 200 patients; systolic × pulse lead => even strong single features still interact.

Post-workshop

- Materials are at **bioinformaticsdotca.github.io** forever(ish)
- Lecture recordings will be uploaded to our Youtube channel and linked on Github
- For those who agreed to share contact info, a mailing list will be sent out
- Certificate will be emailed to you
- Slack channel is available (history expires after 3 months)
- Please keep in touch!

Coffee Break & Networking Session

Workshop Sponsors